Reviving, reproducing, and revisiting Axelrod's second tournament

Vincent Knight[1, *], Owen Campbell[2], Marc Harper[3], T. J. Gaffney[4], and Nikoleta E. Glynatsi[5, 6]

[1]School of Mathematics, Cardiff University, United Kingdom
[2]Independent Researcher, Cardiff, United Kingdom
[3]Google Inc., California, USA
[4]Independent Research, Nevada, USA
[5]Discrete Event Simulations Teams, RIKEN Center for Computational Science, Kobe, Japan
[6]Mathematical Social Science Team, RIKEN Center for Interdisciplinary Theoretical and Mathematical Sciences, Wako, Japan
[*]Corresponding author: KnightVA@cardiff.ac.uk.

Abstract

Direct reciprocity, typically studied using the Iterated Prisoner's Dilemma (*IPD*), is central to explaining cooperation. In the 1980s Robert Axelrod ran two computer tournaments in which Tit-for-Tat (*TFT*) emerged as the winner. Yet the archival record is incomplete: for the first tournament only a report survives, and for the second the submitted Fortran strategies remain but not the final tournament code. We recreate the second tournament by restoring the surviving Fortran implementations to compile with modern compilers and by building an interface that calls the original strategy functions without modification. We reproduce Axelrod's main findings: *TFT* prevails, and successful play tends to be cooperative, responsive to defection, and willing to forgive. Strategy rankings remain mostly unchanged. We then enlarge the field with additional strategies and one of the largest *IPD* tournaments to date, showing the original setting favored *TFT* and that several lesser-known submissions perform strongly in more diverse settings and under noise.

keywords: iterated prisoner's dilemma, evolution of cooperation, replication study, reproducible social science.

Main

Cooperation is fundamental to the organization of societies, yet its persistence remains a central puzzle in evolutionary biology and the social sciences. In the absence of supporting mechanisms, natural selection favors defection over cooperation. To explain the emergence of cooperative behavior, several mechanisms have been proposed, among which *direct reciprocity* has played a central role [1–5]. Direct reciprocity relies on repeated interactions among the same individuals, where decisions can be conditioned on a partner's previous behavior. By rewarding cooperation and punishing defection, such interactions provide a pathway for cooperation to be maintained.

The standard model for direct reciprocity is the *Iterated Prisoner's Dilemma (IPD)*. In each round, two players independently decide whether to cooperate (C) or defect (D). Mutual cooperation yields the reward payoff (R), mutual defection gives the punishment payoff (P), and unilateral defection results in the temptation payoff (T) for the defector and the sucker's payoff (S) for the cooperator. With $T > R > P > S$ and $2R > T + S$, the dilemma arises: defection dominates in the one-shot game, even though mutual cooperation would maximize collective welfare.

Because repeated interactions allow for a wide variety of conditional behaviors, direct reciprocity gives rise to a diverse set of strategies. Much of the field has therefore focused on understanding which strategies are "successful" and under what conditions. Researchers have examined which strategies can constitute Nash equilibria [6–9], which can emerge and persist under evolutionary dynamics [3, 10–14], and which perform best in computational tournaments [15–20]. Studies have also sought to identify the principles underlying successful strategies, such as the balance between retaliation and forgiveness, the role of stochasticity [11], or the capacity to adapt to diverse opponents [21].

A pioneering contribution to the study of direct reciprocity and strategies came from Robert Axelrod's computer tournaments in the 1980s [22–24]. Axelrod invited both academics and the wider public, including readers of computer hobbyist magazines, to submit computer programs for playing the *IPD*. Each submission entered a *round-robin tournament*, facing every other entry, a copy of itself, and a random baseline strategy that cooperated with probability 1/2. Average payoffs across all matches determined the final ranking of strategies. The first tournament [22] attracted 14 entries and was won by *Tit for Tat (TFT)*, a remarkably simple strategy that mirrored the opponent's previous move. A second, larger tournament [23] followed, with 63 strategies this time. The main change of the second tournament was that matches no longer had a fixed length but instead had 5 different lengths unknown to the players, removing the possibility of exploiting a known horizon. Despite the new list of opponents, *TFT* again emerged as the winner. Axelrod argued that its success rested on five key properties: it was *nice* (never the first to defect), *provocable* (ready to retaliate), *forgiving* (able to return to cooperation), *non-envious* (not striving to outperform the opponent), and *simple* (easy to understand and implement).

These pioneering tournaments shaped decades of research on cooperation, yet the archival record surrounding them is incomplete. For the first tournament, only the published report survives; the original source code has been lost. For the second tournament, the Fortran implementation of the submitted strategies remains available online*, but the final code used in Axelrod's reported tournaments has not been preserved.

This highlights a broader challenge of software reproducibility in the *IPD* literature. Over the past 50 years, hundreds of strategies have been proposed, often defined only by fragments of source code, incomplete descriptions, or insufficient supporting data. This has made it difficult to replicate earlier findings, and has led many researchers to evaluate new strategies against small, unrepresentative sets of opponents. Such practices bias conclusions and weaken claims about the relative performance of new strategies. An important step toward addressing this issue has been the `Axelrod-Python` project [25], an open-source Python package that provides a comprehensive framework for implementing and testing *IPD* strategies. The library includes a wide variety of strategies from the literature, together with detailed documentation and usage examples. By providing open, executable implementations, `Axelrod-Python` makes it possible to test strategies under common conditions and compare their performance systematically, and it has therefore been used in ongoing research [19, 21, 26–28].

One fundamental question has remained: *can Axelrod's original results be reproduced?* Addressing this question is important not only for assessing the robustness of foundational research on cooperation, but also as a case study in the reproducibility of computational social science more broadly. In this work, we build on the `Axelrod-Python` project to replicate, as faithfully as possible, the outcomes of Axelrod's *second tournament*. Rather than manually re-implementing the strategies from partial descriptions, which would risk introducing errors, we update the surviving Fortran implementations so that they compile with modern compilers, construct an interface that allows them to interact with Python, and integrate them directly with the `Axelrod-Python` library. This design allows us to call the original functions without modification.

Although Axelrod's reported results are impossible to replicate, we are able to recover the main conclusions of Axelrod's study. At the same time, we extend the analysis by incorporating additional strategies from the `Axelrod-Python` library and ultimately conducting one of the largest *IPD* tournaments to date. These extensions shed new light on the robustness

*See `http://www-personal.umich.edu/~axe/research/Software/CC/CC2/TourExec1.1.f.html` accessed on 2025-09-30.

of the original findings. Overall, this work makes three contributions. First, it provides the first systematic attempt to reproduce the results of Axelrod's second tournament, which have been central to the study of cooperation. Second, it offers a contemporary perspective on Axelrod's experiments. Finally, by reviving Axelrod's original code and making it accessible through the `Axelrod-Python` library, we preserve for future use the strategies of his second tournament.

Results

Reviving the tournament. Axelrod's original tournaments cannot be fully replicated. For the first tournament, the only surviving record is the published report; the source code has been lost. For the second tournament, however, the implementations of the 63 submitted strategies remain available. As an example, the Fortran implementation of *TFT*, named `k92r.f`, is shown in Fig. 1**a**. In this work, we revive the second tournament using these original implementations rather than re-implementing them. To this end, we updated the surviving Fortran code to compile with modern compilers and connected it to the `Axelrod-Python` framework via a Python interface that calls the original functions. The workflow is: (i) build a shared library of the historical strategies; (ii) use the adapter to format the match state, invoke the Fortran routine, and return the action; (iii) let `Axelrod-Python` run matches and assemble tournaments (Fig. 1**b**,**c**). For details on recovering, packaging, and releasing the original source code, see **Methods**.

The major advantage of this approach is that no subjective reinterpretation was introduced. The only modifications were minimal adjustments for compiler compatibility. For several strategies, the Fortran source code is the only surviving description; these strategies are therefore run and evaluated exactly as originally written. This constitutes the most faithful reproduction of Axelrod's work to date.

Reproducing Axelrod's second tournament.

Running the tournament. Axelrod's second tournament included 63 strategies. While Axelrod named a few explicitly, most were referred to only by their ranking in the original tournament or by the names of the individuals who submitted them. We now have access to the function names of the Fortran implementations. In what follows, we refer to strategies by their function names, supplemented by Axelrod's names when available. A complete list of the strategies, their authors, their original rankings, the names provided by Axelrod (where available), and their function names is given in the **Supplementary Information** (S1).

Axelrod use the same payoff matrix as in the first tournament, with $T = 5$, $R = 3$, $P = 1$, and $S = 0$. Unlike the first tournament, players were not informed of the number of rounds in advance. In [23] it is stated that:

> *"As announced in the rules, the length of the games was determined probabilistically with a .00346 chance of ending with each given move. This parameter was chosen so that the expected median length of a game would be 200 moves. In practice, each pair of players was matched five times, and the lengths of these five games were determined once and for all by drawing a random sample. The resulting random sample from the implied distribution specified that the five games for each pair of players would be of lengths 63, 77, 151, and 308 moves. Thus the average length of a game turned out to be somewhat shorter than expected at 151 moves."*

Thus, termination was not determined probabilistically during play. Instead, Axelrod sampled five match lengths in advance of the tournament and applied the same set to every pair of players. The text lists only four lengths $(63, 77, 151, 308)$ even though five games were played. Using the reported average length of 151 moves, the missing length can be inferred to be 156, so we assume the complete set of match lengths to be $\{63, 77, 151, 156, 308\}$.

It also appears that the only mechanism for averaging out stochastic variation in the original setup was the use of these five fixed repetitions. Without access to the random seed used by Axelrod, it is not possible to reproduce the tournament

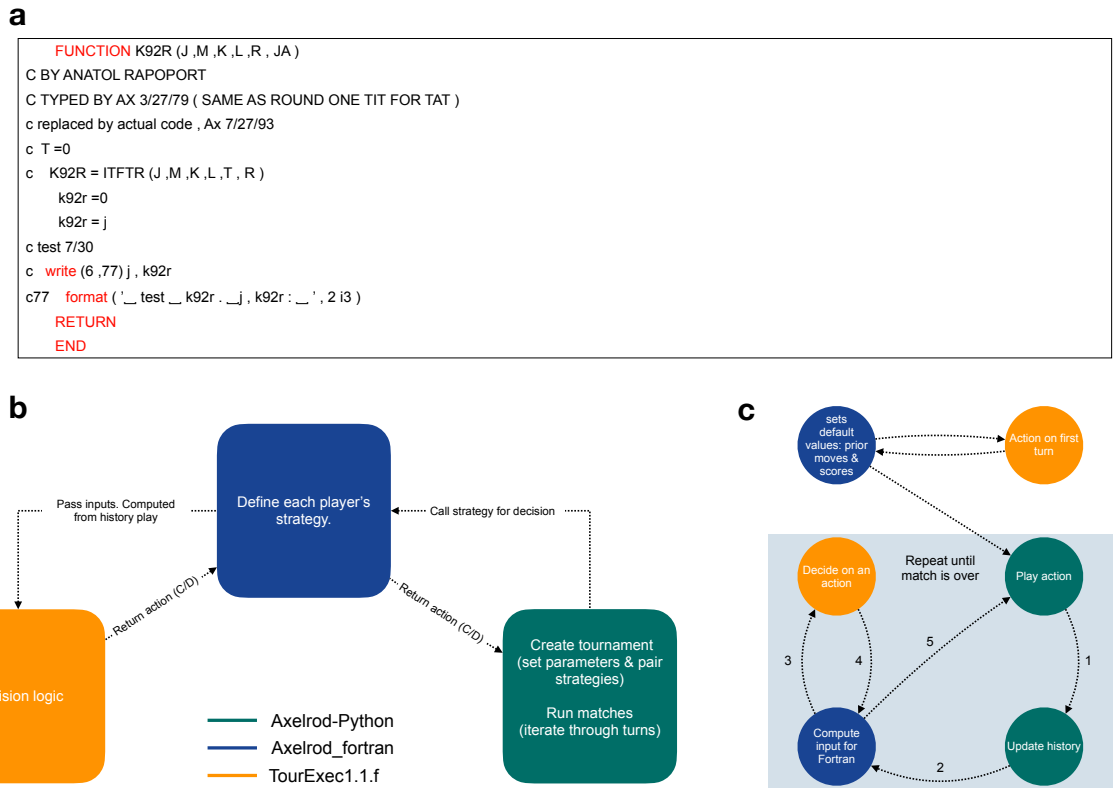


Fig. 1: **Reviving Axelrod's second tournament code. a,** Fortran code for *TFT* (`k92r.f`). Each strategy takes as input the match state (`J`, `M`, `K`, `L`, `R`, `JA`) and returns 0 (cooperate) or 1 (defect). **b,** Overview of how the three codebases interact.`Axelrod_fortran` creates strategy instances for each player. Tournaments and matches are managed by `Axelrod-Python`, which at each turn calls the corresponding `Axelrod_fortran` instance; this in turn invokes the original Fortran code to produce a decision from the match history. **c,** Turn-by-turn execution. On the first turn, `Axelrod_fortran` initializes the parameters required by the Fortran implementation, queries it for a decision, and returns the move to `Axelrod-Python`, which plays it and updates the history. On each subsequent turn: (1) the previous outcome is recorded and the history updated; (2) `Axelrod_fortran` accesses the updated history; (3) formats it into the inputs expected by the Fortran code; (4) calls the Fortran function and receives a decision; (5) `Axelrod-Python` plays the decision. The process repeats until the match ends.

exactly. To obtain smoother estimates of each strategy's performance, we run Axelrod's second tournament a total of 25,000 times. For each run of the tournament, we use the same five match lengths $(63, 77, 151, 156, 308)$ for every pairing of strategies.

The winners, the losers, and the differences. In Fig. 2**a**, we compare the original rankings with those obtained in our replication, with strategies sorted according to their original ranks. As we can see, the winner of the tournament remains *TFT*, while the strategy `k36r`, authored by Roger Hotz, again ranks last. The overall outcomes of most strategies are reproduced, though some do not retain their exact rankings. Notably, the bottom ranking strategies are replicated quite well, consistently performing poorly in both tournaments. Fig. 2**b** shows the difference in rank between the reproduced and original tournaments, ordered from the most negative to the most positive change. We find that 40 out of 63 strategies exhibit a shift in ranking. Most of these changes are modest, typically within 1-3 positions, though two strategies move by 10 places.

These two strategies are `k76r` (originally ranked 46th, now 36th) and `k61r` (originally 2nd, now 12th). The latter, `k61r`, is also known as *Champion*, after its author's surname. Champion cooperates for the first ten moves, plays *TFT* for the next fifteen, and thereafter cooperates unless (i) the opponent defected on the previous move, (ii) the opponent's overall cooperation rate falls below 60%, and (iii) a random draw in $[0, 1]$ exceeds the opponent's cooperation rate. Upon closer inspection, we identified a bug in the Fortran implementation of Champion: an internal variable was not properly initialized. Specifically, the variable `ICOOP` was not assigned an initial value within the function. On the very first call `ICOOP` is effectively treated as 0, but in later matches this assumption no longer holds, meaning that while in the first match `ICOOP` starts at 0 and increases as expected, in subsequent matches its (undefined) value can persist across calls. The corrected version of this strategy is shown in **Extended Data Fig. 1**. It is unclear whether this bug affected the tournament results reported in 1980. One possibility is that each match was run in isolation, which would have prevented the error from arising. In our analysis we use the corrected version of the strategy.

In Fig. 2**c**, we examine the average scores of the strategies. Champion shows a noticeable drop relative to the now second and third ranked strategies. However, its mean score remains very close to that of the rest of the top ten (see **Extended Data Table 1**). Fig. 2**d** presents violin plots of the score distributions for these top 15 strategies across the 25,000 repetitions. Champion rarely outperforms the second or third ranked strategies, doing so in only a single tournament, confirming that, at least in this version, Champion is not as strong as originally reported.

An interesting observation is that strategies such as `k32r` and `k75r` occasionally achieve the maximum possible score of 2.9. Across all repetitions, these strategies won 0.8% and 2.3% of the tournaments, respectively (Table 1). They received little attention in the original tournament, likely because their high variance obscured their potential relative to consistently strong performers such as *TFT*. Another strong and comparatively stable strategy is `k42r`, which won 31.5% of our replicated tournaments; for comparison, *TFT* won 63.3% (Table 1). Because the original tournament is a single realization, a different random seed could plausibly have produced a different winner: namely, on another roll of the dice, `k42r` (Otto Borufsen) might well have been crowned the winner.

The source code for `k42r`, together with a description of the strategy's behaviour, is provided in the **Supplementary Information** (S6). In summary, the strategy has two states: *normal* and *defecting*. In its normal state, it behaves much like *TFT*, but it includes mechanisms to recover from mutual defection and to escape alternating defections. Every 25 turns, it evaluates the opponent's cooperation rate and switches to defecting mode if the opponent appears random or highly defective. The `k42r` strategy can thus be viewed as a more sophisticated and adaptive variant of *TFT*. These mechanisms are likely intended to prevent endless retaliation and to guard against exploitation by less cooperative opponents. However, in Axelrod's original tournament, `k42r` ranked only 3th, suggesting that these mechanisms were not particularly advantageous/or used against the original pool of strategies and highly cooperative environment.

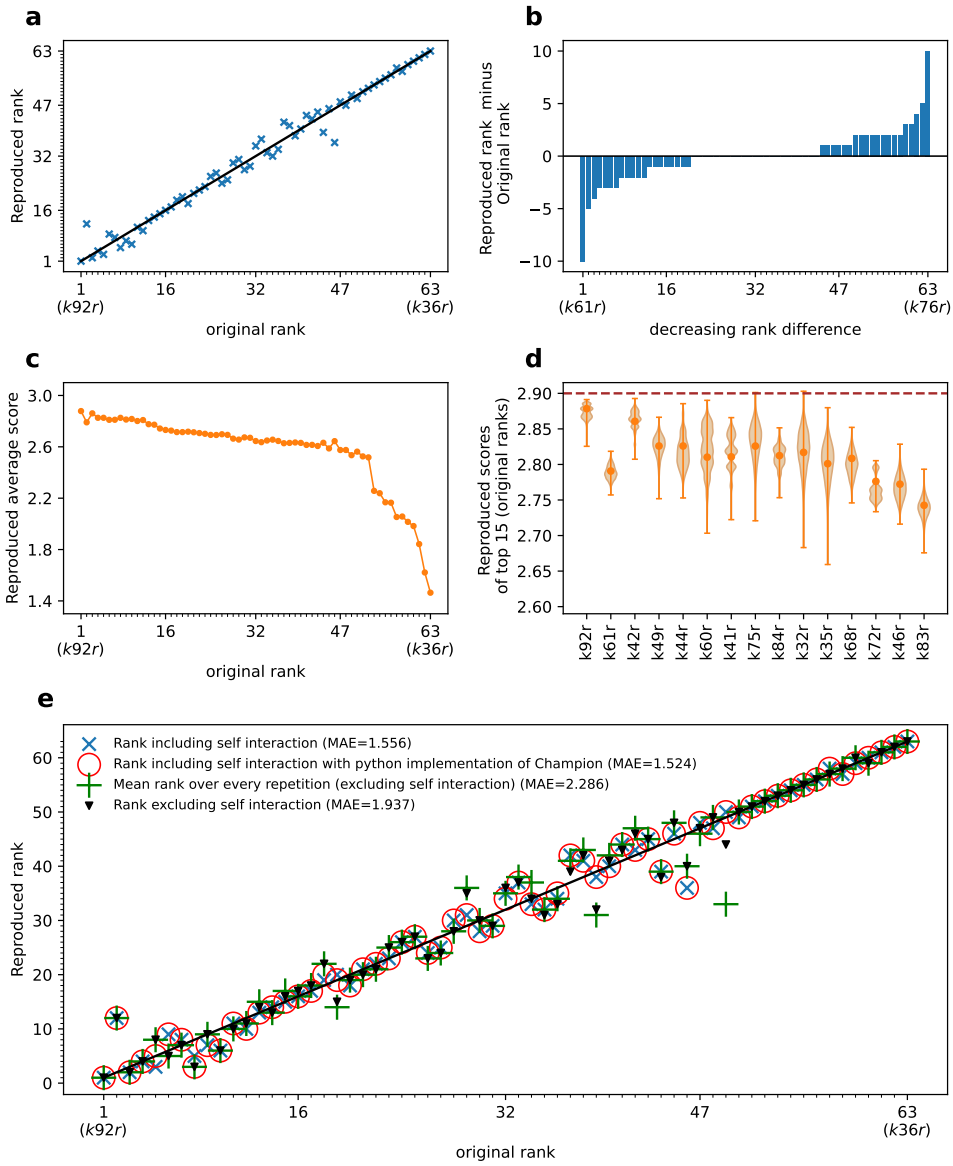


Fig. 2: **Reproducing Axelrod's second tournament. a,** Comparison of the original rankings and the replicated rankings. Strategies are ordered by their original ranks; *TFT* again emerges as the overall winner, while `k36r` again ranks last. **b,** Differences in rank between the original and reproduced tournaments, ordered from most negative to most positive change. Most strategies shift by only 1-3 positions, though a few exhibit larger changes, including Champion (`k61r`). **c,** Mean scores of all strategies. Champion's average score drops relative to the other three strategies but remains close to the rest of the top ten. **d,** Violin plots showing the distribution of scores for the top 15 strategies across all repetitions. **e,** Results under alternative replication approaches, including with and without self-interactions and using a Python re-implementation of Champion. Across all settings, *TFT* remains the winner, while Champion exhibits the largest change in ranking.

	Win proportion
k32r	0.0079
k35r	0.0002
k42r	0.3136
k44r	0.0038
k60r	0.0103
k75r	0.0226
k78r	0.0016
k82r	0.0083
k92r	0.6319

Table 1: **The proportion of reproduced tournaments won by each strategy.** The table shows all strategies who won one of the repetitions of the original tournament. *TFT* wins 63.3% of them indicating that 36.7% of used random seeds lead to a different winner.

We also explored alternative approaches in our attempt to replicate the original ranking (Fig. 2**e**). These include running the tournament with and without self-interactions, as well as testing a Python implementation of Champion that we developed from the textual description in [23]. Across all approaches, Champion consistently exhibits the largest change in ranking among the top strategies, while *TFT* remains the overall winner.

Overall, the minor discrepancies in rankings and scores can be attributed to one or more of the following:

- An unknown modification of the source code of Champion (`k61r`) used in the final original tournament code [23].
- Stochastic variation not being sufficiently taken in to account in the original tournament [23].
- A difference with how an older Fortran compiler would interpret the commands, though the implemented version seems to interact as expected.

The representative strategies. Axelrod conducted an analysis to identify what he termed "representative strategies" [23]. These were strategies that, according to a linear regression model, served as strong predictors of overall performance (average score). The representative strategies identified in the original tournament were S_6, Reversed State Transition, Ruler, Tester, and Tranquilizer. Using the scores against these five strategies as predictors, Axelrod reported an R^2 value of 0.979, indicating that 97% of the variance in overall performance could be explained by the model. For completeness, the coefficients of this regression (as reported in [23]) are listed in **Extended Data Fig. 2**.

We replicated Axelrod's linear regression analysis and evaluated the predictive power of the original model using our reproduced tournament data. We obtained an R^2 of 0.957, demonstrating that the model continues to be a strong predictor of performance despite the small discrepancies in individual rankings.

To further assess the robustness of the model, we performed a backward elimination procedure to examine how R^2 changes as the number of predictor strategies increases. We find that the largest gain occurs when five predictors are included, after which improvements are small (**Extended Data Fig. 2**.). This suggests that Axelrod's choice of five representative strategies was reasonable. Whether this selection was the outcome of a systematic feature selection procedure or a judgment call remains unclear; in either case, it appears well chosen.

Cooperation in the original tournament. Here we ask: *what was the original tournament like?* The answer is that it was, in fact, highly cooperative. The mean cooperation rate across the tournaments is 75%. When we plot cooperation rates against the original rankings (Fig. 3**a**), most strategies are indeed highly cooperative, with the exception of those in the bottom 14% of the ranking, which display cooperation rates below 0.5.

In Fig. 3**b**, we show that the top performing strategies cooperate almost perfectly with one another, one of Axelrod's key observations. He also emphasized the importance of *provocability* (willingness to punish defection) and *forgiveness* (ability to return to cooperation). In the case of *TFT*, this means responding immediately to the opponent's last move: *TFT* retaliates after defection and resumes cooperation after cooperation. As a result, *TFT* achieves the same cooperation rate as its opponent. To capture this notion, in Fig. 3**c** we plot the difference between pairwise cooperation rates and their transposes, that is, the deviation from symmetry. A value of 0 indicates that two strategies cooperate at exactly the same rate with one another. The top performing strategies cluster near 0, whereas lower performing strategies are more asymmetric, with larger positive or negative deviations.

These outcomes are not surprising. By the time of the second tournament, Axelrod's earlier results were already known, and the strong performance of *TFT* was widely recognized. As a consequence, many newly submitted strategies were explicitly designed to resemble *TFT* or to cooperate effectively with *TFT*-like rules, hence their high levels of cooperation and the symmetry in their interactions.

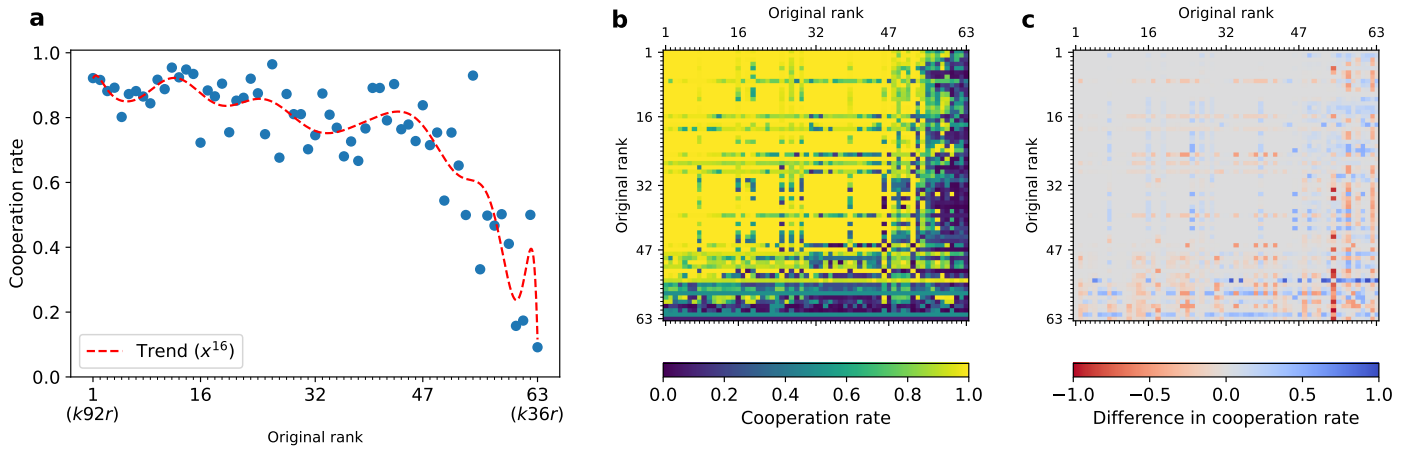


Fig. 3: **Cooperation in Axelrod's second tournament. a,** Mean cooperation rate by original ranking. Most strategies are highly cooperative, with the exception of those at the bottom of the ranking, which fall below 50%. **b,** Cooperation rates between each pair of players, players are ordered by their original ranking. **c,** Deviation from symmetry in pairwise cooperation rates, defined as the difference between cooperation matrices and their transposes. Values close to zero indicate that both strategies cooperate with one another at the same rate.

Revisiting the tournament.

Extra invitations. We investigate how the results would change if Axelrod had received additional submissions. The `Axelrod-Python` library includes 209 strategies that did not appear in the original tournament. We therefore examine the effect of including new strategies. We do this by first simulating a tournament with all 272 strategies repeating each match-length 250 times. This reduced number of repetitions was due to the combinatorial difficulty of running such a large tournament. This full tournament will be discussed subsequently. We use the interactions from this full tournament to see the effect of first adding a single new strategy, then considering all possible pairs, triples, and quadruples drawn from the remaining pool. This yields a total of 78,760,605 tournaments:

1. Adding 1 strategy: $\binom{209}{1} = 209$ tournaments.
2. Adding 2 strategies: $\binom{209}{2} = 21{,}736$ tournaments.
3. Adding 3 strategies: $\binom{209}{3} = 1{,}499{,}748$ tournaments.

4. Adding 4 strategies: $\binom{209}{4} = 77{,}238{,}876$ tournaments.

Our analysis focuses on how the additional submissions affect the performance of the *original* 63 strategies. For the original strategies that win at least one of these new tournaments, we show the probability of winning the tournament when 1, 2, 3, or 4 additional strategies are included (Table 2). These include *TFT*, and high-scoring strategies such as `k75r`, `k32r`, and `k42r`.

A striking pattern is that the effect of extra invitations on *TFT* is *non-monotonic*. Adding a single additional invitee substantially *increases* the probability that *TFT* wins (to about 85%); with two invitees the probability is lower (∼71%), with four invitees it is higher again (∼75%), and only with three invitees do we observe a drop (to ∼60%). Overall, these results highlight the robustness of *TFT* in Axelrod's second tournament: it is difficult to unseat as the top performer. The next most successful strategy, `k42r`, remains competitive (e.g., winning up to ∼37% with three additional invitees), whereas strategies such as `k32r` and `k75r`, which can achieve high mean scores in some realizations, tend to be even less effective as more entrants are introduced.

	1 New (N = 209)	2 New (N = 21736)	3 New (N = 1499784)	4 New (N = 77238876)
k32r	0.00000	0.00000	0.00001	0.00079
k41r	0.00000	0.00000	0.00001	0.00000
k42r	0.14833	0.26941	0.36640	0.21723
k44r	0.00000	0.00023	0.00057	0.00461
k49r	0.00000	0.00014	0.00035	0.00023
k60r	0.00478	0.01118	0.01882	0.00451
k75r	0.00000	0.00051	0.00245	0.00086
k92r	0.84689	0.71747	0.60814	0.75412
Sum	1.00000	0.99894	0.99674	0.98235

Table 2: **Proportion of wins for original strategies with 1-4 additional invitees.** Shown are the strategies with non zero probabilities of winning at least one of the modified tournaments under five fixed match lengths, repeated 250 times. Additional invitees are strategies drawn from the open-source `Axelrod-Python` library; a full list is provided in the **Supplementary Information**, and source code/documentation are referenced therein. We consider four cases: adding 1, 2, 3, or 4 extra strategies to the original field. The last row reports the sum across these strategies (noting that some wins accrue to strategies not submitted to the original tournament).

A noisy tournament. A natural question is how the results would change if the original tournament had included *noise*, i.e., a small probability that a player's intended action is flipped. Noise was among the main criticisms of Axelrod's study, since *TFT* is not robust to accidental defections [29–32].

Fig. 4 summarizes the results for a noise probability of 0.01. Fig. 4**a** shows rank changes among the 63 original submissions: the left column orders strategies by their original rank, while the right column lists ranks 1-63, with arrows indicating each strategy's rank in the noisy tournament. As expected, noise leads to substantial reordering, with several mid ranking strategies moving into top positions and some former top performers dropping. The mean cooperation rate declines from 0.75 (noiseless) to 0.65 under noise (Fig. 4**c**). Correspondingly, the new winning strategy in this setting cooperates at 0.61, slightly *below* the noisy-tournament mean, whereas *TFT* cooperates at 0.70, *above* the mean. This pattern reinforces the idea that *cooperating slightly less than the tournament average* is a good predictor of success when noise is present [21]. Figures 4**d** and **e** show that pairwise cooperation becomes more asymmetric and generally lower under noise. Strategies that tend to cooperate less than their co-players perform better. More complex strategies with additional mechanisms also fare well; for example, `k42r`, now ranked 10th, outperforms *TFT*.

We emphasize that evaluating the original submissions under noise is not entirely fair: many authors would likely have

incorporated explicit error-correction or forgiveness mechanisms had noise been part of the original rules.

Large tournaments. In this section, we examine how the original strategies perform in larger and more diverse tournaments. First, we combine the original submissions with the strategies from the Stewart and Plotkin (S & P) tournament [16]. Next, we run three large-scale tournaments using the full `Axelrod-Python` library under three noise regimes: no noise, 1% noise, and 5% noise. We report the outcomes for each tournament separately. Full results, including all rankings and cooperation rates, are provided in the **Supplementary Information**.

Since the work of Press and Dyson [33], there has been sustained interest in zero-determinant (ZD) strategies. Stewart and Plotkin [16] presented a focused tournament pitting ZD strategies against a small collection of other rules. When all S & P strategies are added to Axelrod's second tournament, the resulting relative rank changes for the original strategies are modest (Fig. 5). The overall level of cooperation remains high (mean ≈ 0.73), and the leading strategies are highly cooperative with one another, mirroring the qualitative pattern of the original tournament. **Extended Data Table 2** lists the top-performing strategies (see the **Supplementary Information** (S3) for the full ranking). In this new tournament, ZD strategies are distributed across the ranking rather than concentrated at the top (**Supplementary Information** (S3)), while *TFT*, `k42r`, and generous variants (e.g., GTFT [34] and ZD-GTFT-2) perform particularly well.

The `Axelrod-Python` library contains more than 200 strategies and regularly hosts the largest *IPD* tournaments to date (each time a new strategy is contributed, the results are updated). We expand this tournament further by including all of the original Fortran strategies (omitting the Python duplicates). This produces a tournament with 272 strategies. **Extended Data Table 3** shows the top 16 strategies of the tournament.

Rank changes relative to the reproduced tournament are shown in Fig. 6**a**. Several mid performing strategies from the original tournament rise into top positions in this more complex environment. Notably, `k42r` performs best among the original strategies, ranking 16th overall. The cooperation rate of the library-only tournament (excluding the original Fortran strategies) is 0.619. By contrast, the overall cooperation rate of the expanded tournament that includes the Fortran strategies is 0.626 (see Fig. 6**c**).

Pairwise interactions are shown in Figs. 6**d**–**e**, where top strategies are seen to cooperate strongly with one another, but do not when facing lower ranking opponents. The new winners appear to exploit weaker strategies, which contributes to their success. The highest ranking strategies overall are sophisticated learners trained via reinforcement learning. The "Evolved" strategies were trained with evolutionary algorithms, while the "PSO" strategies were trained using particle swarm optimisation, as described in [26]. These high performing strategies do not necessarily follow Axelrod's original guidelines for success. Instead, they align more closely with the principles identified in [21], which are as follows:

1. Be a little envious.
2. Be "nice" in non-noisy environments or when games are long (if known).
3. Reciprocate both cooperation and defection appropriately: be provocable in short matches and generous in noisy settings.
4. It is acceptable to be clever.
5. Adapt to the environment; adjust to the mean tournament cooperation level.

Finally, we consider the `Axelrod-Python` tournament with noise, using noise probabilities of 1% and 5%. The results are shown in **Extended Data Figs. 3–4**. As expected, the overall cooperation rate is lower than in the noiseless, reproduced, and reproduced with noise tournaments. Notably, several of the original submissions perform very well: the

best performer from the original set ranks 5th with 1% noise and 3rd with 5% noise. This suggests that some of the more sophisticated strategies submitted to Axelrod's original study were perhaps too complex for the highly cooperative, relatively stable conditions where "cooperate, retaliate after defection, forgive after cooperation, and avoid exploiting others" was optimal. In the presence of errors, and against more complex competitors, these strategies make use of their additional mechanisms to maintain competitive payoffs and robust rankings.

Conclusion

Our study revisits one of the most influential computational experiments in the social sciences: Axelrod's second Iterated Prisoner's Dilemma (*IPD*) tournament. By using the original Fortran implementations available online and ensuring that they still compile, we provide the first systematic reproduction of Axelrod's results. Despite minor discrepancies, most notably a bug we identified in *Champion* (`k61r`), our analyses confirm the robustness of Tit For Tat (*TFT*) as a leading strategy. This reinforces Axelrod's conclusion that reciprocity, moderated by the capacity for retaliation and forgiveness, is an effective principle for sustaining cooperation.

By expanding the tournament to include additional strategies from the literature, we reveal clearer patterns. Namely, *TFT* is remarkably robust and difficult to displace when the majority of opponents are drawn from the original pool. In other words, Axelrod's tournament, and similar ones, such as those organized by Stewart and Plotkin [16], create highly cooperative environments in which top performing strategies tend to reciprocate with one another, conditions that are especially favorable to *TFT*.

The picture changes in more diverse or noisy contexts. Several strategies that ranked only moderately in Axelrod's original results rise substantially under increased complexity or noise. A prominent example is `k42r`, which wins 35% of replications of the original tournament in our experiments. This strategy outperforms *TFT* in the presence of noise and surpasses any of the original submissions in tournaments with more elaborate rule sets, such as the full `Axelrod-Python` set of 209 strategies. Other mid performing strategies from the original tournament likewise move into top positions in these settings, particularly when noise is introduced. These findings suggest that some sophisticated designs may have been "over-engineered" for the cooperative, relatively stable conditions of the original experiment, yet prove better adapted to noisy or diverse environments.

This observation aligns with a broader literature showing that *TFT* is not universally optimal. Variants such as Win–Stay–Lose–Shift [35], Generous *TFT* [34], and "All-or-nothing" [13] strategies, as well as "friendly rivals" [7] and strategies discovered via reinforcement learning [19], can outperform *TFT*. Our contribution is to show that, even within Axelrod's original submission set, strategies that received little attention due to modest performance in the original tournament can excel when the environment is more diverse, or noisier. These results also help revise the common impression, shaped by the original tournament, that *TFT* is universally the best performing strategy.

A second contribution of this work is methodological. To our knowledge, this is the first effort to package and reproduce, according to contemporary best practices, code originally written in the 1980s. The archived materials [36] (at `https://doi.org/10.5281/zenodo.17250038`) are curated to high standards of reproducible research [37] and accompanied by a fully automated test suite. All changes to the original code were made systematically and transparently, with complete records available at (`https://github.com/Axelrod-Python/axelrod-fortran`). More broadly, it serves as a case study in computational reproducibility: Axelrod's highly cited, foundational tournaments were built on fragile digital artifacts, of which only fragments survive. Without systematic preservation, much of this historical work might have been lost. By updating, documenting, and integrating the surviving code into an open-source, test-driven framework, we help ensure continued access for the research community.

In conclusion, Axelrod's tournaments remain landmark contributions to the study of cooperation, but their lessons are

more context dependent than is often assumed. While *TFT* is strikingly robust in cooperative environments resembling the original tournament, more complex strategies can outperform it in heterogeneous or noisy settings. We hope that the field will continue to develop preservation pipelines that bring historical computational experiments into alignment with modern standards of reproducibility, and to advance reproducible software and science.

Methods

The surviving code of Axelrod's second tournament was written in Fortran and is hosted online by the University of Michigan Center for the Study of Complex Systems [38], where it was last updated in 1996. The source code for all strategies is contained in a single file, `TourExec1.1.f`, embedded in an HTML page.

Our first step was to ensure that this code could be executed with modern software. We stripped the HTML formatting to recover the raw Fortran source and applied only minimal modifications to enable compilation with contemporary Fortran compilers. Each strategy was then extracted into its own modular file. To streamline the build process, we created a dedicated `Makefile` that compiles all strategy files into a single shared library (`libstrategies.so`). Once installed in a standard location on a POSIX-compliant operating system (e.g., `/usr/local/lib/`), this library is automatically located at runtime. The modified source code is openly available at `https://github.com/Axelrod-Python/TourExec` and has been archived at [39], ensuring long-term accessibility.

Each Fortran strategy was implemented as a function with a fixed set of input arguments that encode the state of the match:

- `J`: Opponent's previous move (0 = cooperate, 1 = defect),
- `M`: Current turn number (starting at 1),
- `K`: Player's cumulative score,
- `L`: Opponent's cumulative score,
- `R`: Random number between 0 and 1 (for stochastic strategies),
- `JA`: Player's previous move.

Given these inputs, the function outputs either 0 (cooperate) or 1 (defect).

With the strategies executable again, the next challenge was to run the tournament. For this we rely on the open-source `Axelrod-Python` library, which provides a modern implementation of *IPD* tournaments. In this framework, a *tournament* is a collection of matches, and each *match* is a repeated game between two strategies. A tournament class manages the overall structure (pairing strategies, scheduling matches, aggregating results), while strategy classes determine actions turn by turn based on the recorded history of play.

To integrate the original Fortran strategies into this environment, we developed a Python package, `Axelrod_fortran` [40]. The `Axelrod_fortran` adapter class inherits from the base `Axelrod-Python` strategy class and translates between the two languages: it constructs the required inputs, calls the Fortran function, and returns the chosen move to the Python environment (Fig. 1). The adapter also handles the initial moves of both players required by the Fortran strategies, which is fixed to cooperation in the original code.

Data availability

The data generated and analysed in this study are available in the Zenodo repository [36] at `https://doi.org/10.5281/zenodo.17250038`.

Code availability

The code used to generate and run the tournaments, as well as all analysis scripts, has been archived alongside the data at the same Zenodo repository and is also accessible via the project's online repository: `https://github.com/Axelrod-Python/revisiting-axelrod-second/`. Instructions for installation and reproduction are provided in the repository README.

Acknowledgements

We thank all contributors to the `Axelrod-Python` library. We also acknowledge the open source ecosystem that enabled this work, including NumPy, pandas, and Matplotlib. This study would not have been possible without the broader open source community.

Author contributions

V.K., O.C., M.H., T.J.G., and N.E.G. designed the study. V.K., O.C., and M.H. wrote the `Axelrod_fortran` software. V.K. performed the analysis. V.K., M.H., and N.E.G. discussed the results and wrote the manuscript.

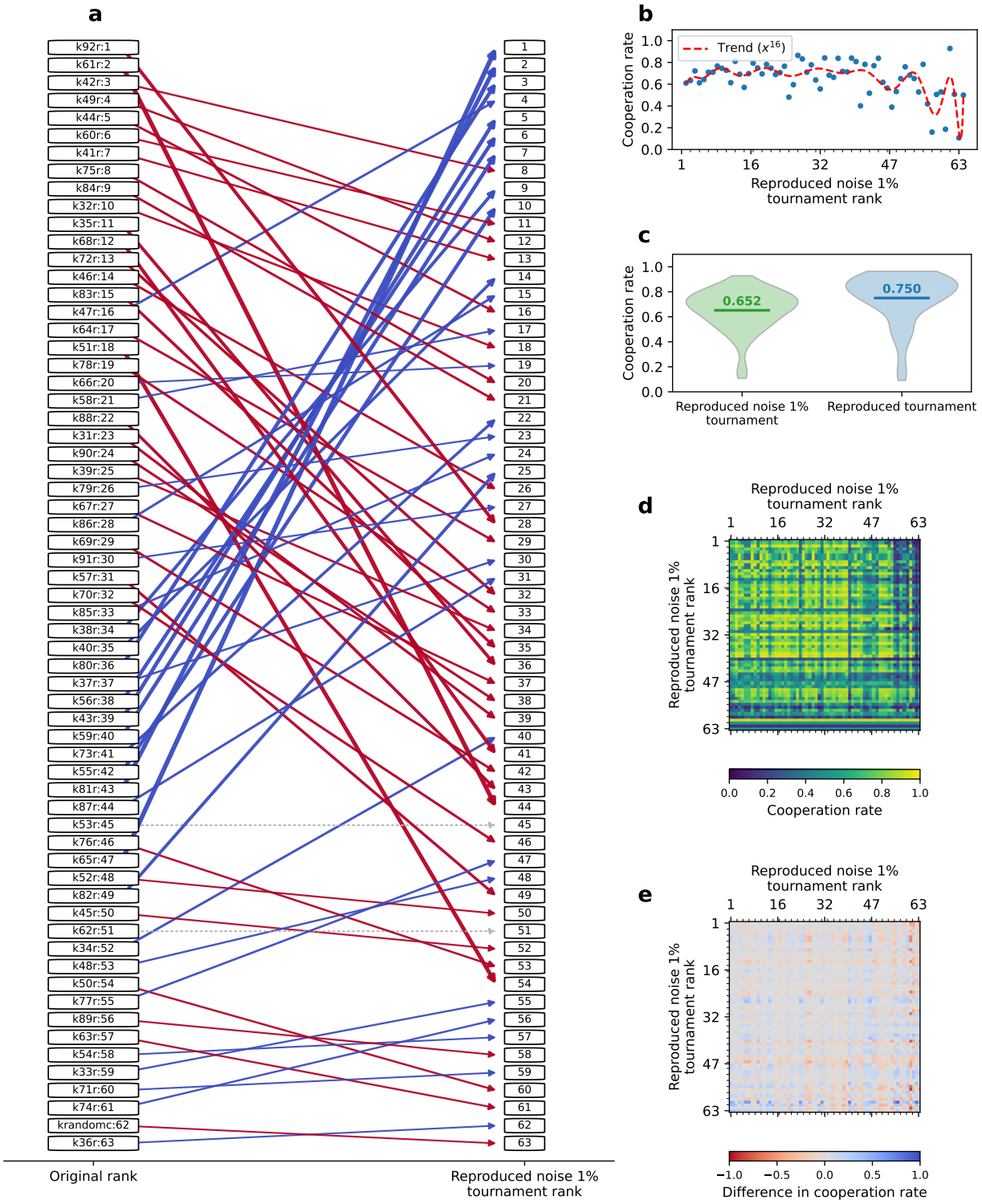


Fig. 4: **Axelrod's second tournament with 1% noise. a,** Comparison of the original rankings and the rankings of the second tournament with noise. In the left column, strategies are ordered by their original ranks, and arrows indicate their ranks in the noisy tournament (right column). The arrow widths are proportional to the change in rank. Blue arrows indicate a positive change in rank, red arrows indicate a negative change in rank, and grey arrows indicate no change in rank. **b,** Cooperation rates of strategies ordered by rank in this tournament. **c,** Tournament cooperation rates across all repetitions for this tournament (green) and the reproduced tournament without noise (blue). We also plot the average cooperation rate (green or blue line) and report its value. **d,** Pairwise cooperation rates between strategies. For each pair we calculate the cooperation rate of the row strategy towards the column strategy and vice versa. The strategies are ordered by the ranks in this tournament. For a full list of strategies see the **Supplementary Information**. **e,** Differences in cooperation rates for each pair of strategies, again ordered by their rank in the reproduced tournament with noise.

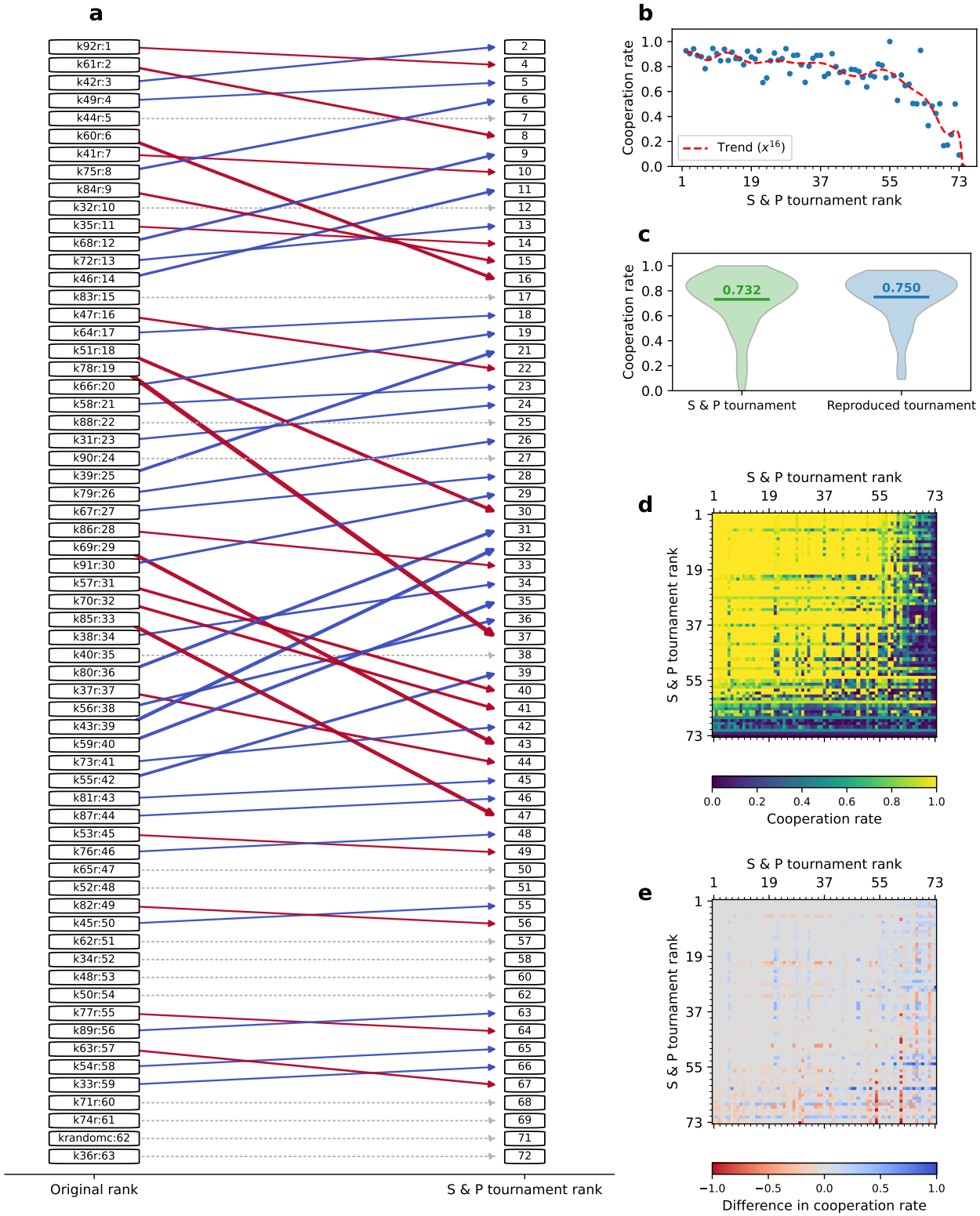


Fig. 5: **Reproduction of Axelrod's second tournament with the addition of the strategies from Stewart's and Plotkin's tournament [16].** Similar to Figure 4. The right column shows the rankings on the original strategies in this new tournament.

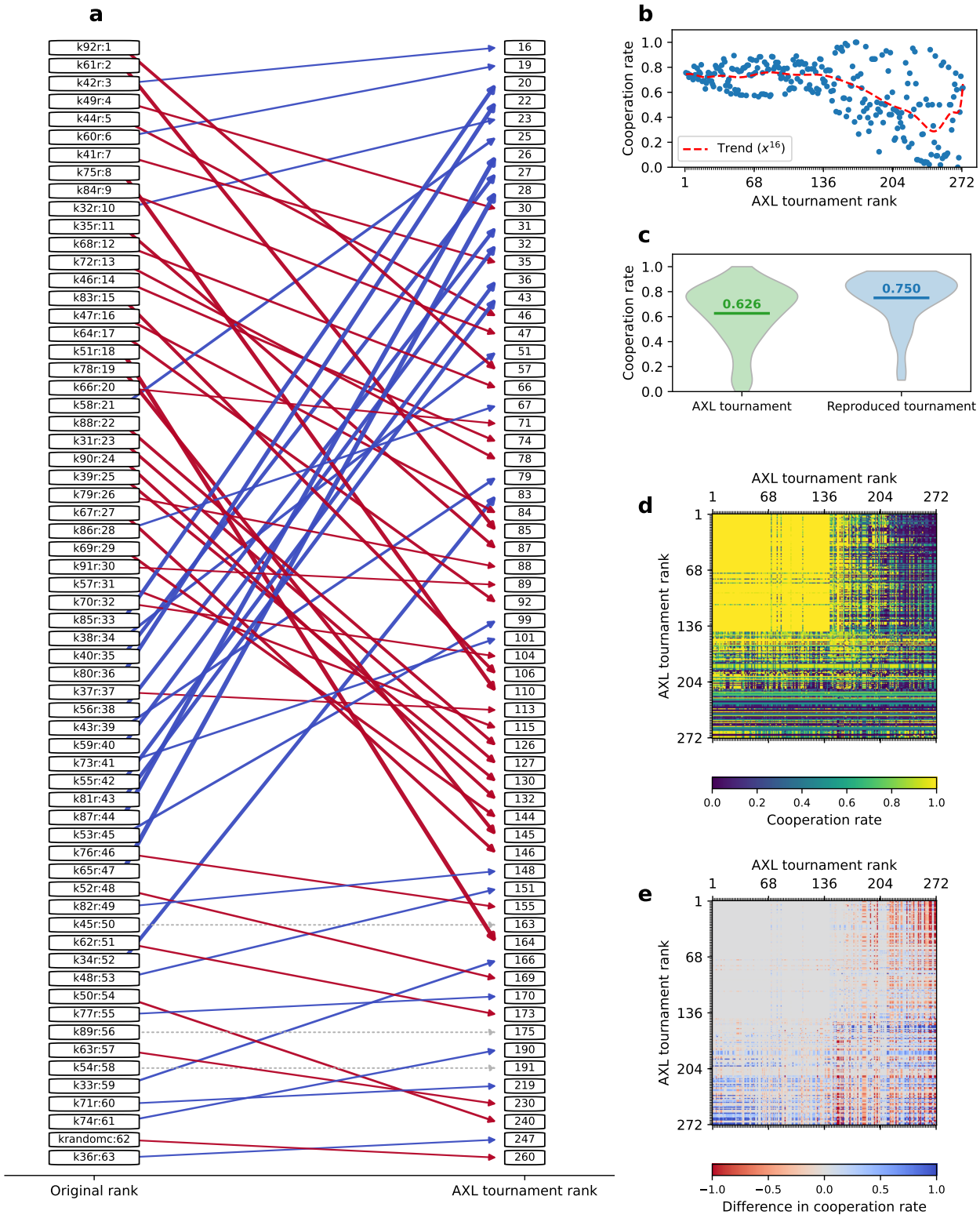


Fig. 6: **Reproduction of Axelrod's second tournament with the addition of the strategies from the `Axelrod-Python`.** Similar to Figure 5.

References

[1] Trivers, R. L. The evolution of reciprocal altruism. *The Quarterly Review of Biology* **46**, 35–57 (1971).

[2] Nowak, M. A. Five rules for the evolution of cooperation. *Science* **314**, 1560–1563 (2006).

[3] García, J. & Van Veelen, M. No strategy can win in the repeated prisoner's dilemma: linking game theory and computer simulations. *Frontiers in Robotics and AI* **5**, 102 (2018).

[4] Glynatsi, N. & Knight, V. A bibliometric study of research topics, collaboration and centrality in the field of the Iterated Prisoner's Dilemma. *Humanities and Social Sciences Communications* **8**, 45 (2021).

[5] Rossetti, C. S. & Hilbe, C. Direct reciprocity among humans. *Ethology* **130**, e13407 (2024).

[6] Hilbe, C., Chatterjee, K. & Nowak, M. A. Partners and rivals in direct reciprocity. *Nature human behaviour* **2**, 469–477 (2018).

[7] Murase, Y. & Baek, S. K. Five rules for friendly rivalry in direct reciprocity. *Scientific reports* **10**, 16904 (2020).

[8] Chen, X. & Fu, F. Outlearning extortioners: unbending strategies can foster reciprocal fairness and cooperation. *PNAS nexus* **2**, pgad176 (2023).

[9] Glynatsi, N. E., Akin, E., Nowak, M. A. & Hilbe, C. Conditional cooperation with longer memory. *Proceedings of the National Academy of Sciences* **121**, e2420125121 (2024).

[10] Van Veelen, M., García, J., Rand, D. G. & Nowak, M. A. Direct reciprocity in structured populations. *Proceedings of the National Academy of Sciences* **109**, 9929–9934 (2012).

[11] Baek, S. K., Jeong, H.-C., Hilbe, C. & Nowak, M. A. Comparing reactive and memory-one strategies of direct reciprocity. *Scientific reports* **6**, 25676 (2016).

[12] Amaral, M. A., Wardil, L., Perc, M. & da Silva, J. K. Stochastic win-stay-lose-shift strategy with dynamic aspirations in evolutionary social dilemmas. *Physical Review E* **94**, 032317 (2016).

[13] Hilbe, C., Martinez-Vaquero, L. A., Chatterjee, K. & Nowak, M. A. Memory-n strategies of direct reciprocity. *Proceedings of the National Academy of Sciences* **114**, 4715–4720 (2017).

[14] LaPorte, P., Hilbe, C. & Nowak, M. A. Adaptive dynamics of memory-one strategies in the repeated donation game. *PLoS Computational Biology* **19**, e1010987 (2023).

[15] Li, J. *et al.* How to design a strategy to win an ipd tournament. *The iterated prisoner's dilemma* **20**, 89–104 (2007).

[16] Stewart, a. J. & Plotkin, J. B. Extortion and cooperation in the Prisoner's Dilemma. *Proceedings of the National Academy of Sciences* **109**, 10134–10135 (2012).

[17] Carvalho, A., Rocha, H., Amaral, F. & Guimaraes, F. Iterated prisoner's dilemma-an extended analysis. *Iterated Prisoner's Dilemma-An extended analysis* (2013).

[18] Gaudesi, M., Piccolo, E., Squillero, G. & Tonda, A. Exploiting evolutionary modeling to prevail in iterated prisoner's dilemma tournaments. *IEEE Transactions on Computational Intelligence and AI in Games* **8**, 288–300 (2015).

[19] Knight, V., Harper, M., Glynatsi, N. E. & Campbell, O. Evolution reinforces cooperation with the emergence of self-recognition mechanisms: An empirical study of strategies in the moran process for the iterated prisoner's dilemma. *PloS one* **13**, e0204981 (2018).

[20] Glynatsi, N. E., Akin, E., Nowak, M. A. & Hilbe, C. Conditional cooperation with longer memory. *Proceedings of the National Academy of Sciences* **121**, e2420125121 (2024).

[21] Glynatsi, N. E., Knight, V. & Harper, M. Properties of winning iterated prisoner's dilemma strategies. *PLOS Computational Biology* **20**, e1012644 (2024).

[22] Axelrod, R. Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution* **24**, 3–25 (1980).

[23] Axelrod, R. More Effective Choice in the Prisoner's Dilemma. *Journal of Conflict Resolution* **24**, 379–403 (1980).

[24] Axelrod, R. M. The evolution of cooperation: revised edition (2006).

[25] The Axelrod project developers. Axelrod-python/axelrod: v3.3.0. https://doi.org/10.5281/zenodo.836439 (2017).

[26] Harper, M. *et al.* Reinforcement learning produces dominant strategies for the iterated prisoner's dilemma. *CoRR* **abs/1707.06307** (2017). URL `http://arxiv.org/abs/1707.06307`.

[27] Bucciarelli, E., Chen, S.-H., Ascatigno, A. & Colantonio, A. Studying the distribution of strategies in the two-scenario snowdrift game. In *Digital (Eco) Systems and Societal Challenges*, 407–428 (Springer, 2024).

[28] Macmillan-Scott, O., Ünver, A. & Musolesi, M. Game-theoretic agent-based modelling of micro-level conflict: Evidence from the isis-kurdish war. *Plos one* **19**, e0297483 (2024).

[29] Molander, P. The optimal level of generosity in a selfish, uncertain environment. *Journal of Conflict Resolution* **29**, 611–618 (1985).

[30] Bendor, J. Uncertainty and the evolution of cooperation. *Journal of Conflict resolution* **37**, 709–734 (1993).

[31] Wu, J. & Axelrod, R. How to cope with noise in the iterated prisoner's dilemma. *Journal of Conflict resolution* **39**, 183–189 (1995).

[32] Do Yi, S., Baek, S. K. & Choi, J.-K. Combination with anti-tit-for-tat remedies problems of tit-for-tat. *Journal of Theoretical Biology* **412**, 1–7 (2017).

[33] Press, W. H. & Dyson, F. J. Iterated Prisoner's Dilemma contains strategies that dominate any evolutionary opponent. *Proceedings of the National Academy of Sciences of the United States of America* **109**, 10409–13 (2012). URL `http://www.pnas.org/content/109/26/10409.abstract`.

[34] Nowak, M. A. & Sigmund, K. Tit for tat in heterogeneous populations. *Nature* **355**, 250–253 (1992).

[35] Nowak, M. & Sigmund, K. A strategy of win-stay, lose-shift that outperforms tit-for-tat in the prisoner's dilemma game. *Nature* **364**, 56–58 (1993).

[36] Knight, V., Campbell, O., Harper, M., Gaffney, T. & Glynatsi, N. E. Source code and data for: Reviving, reproducing, and revisiting axelrod's second tournament. (2025). URL `https://doi.org/10.5281/zenodo.17250038`.

[37] Wilson, G. *et al.* Best practices for scientific computing. *PLOS Biology* **12**, e1001745 (2014).

[38] Axelrod, R. Complexity of cooperation web site. http://www-personal.umich.edu/ axe/research/Software/C-C/CC2.html (1996).

[39] Campbell, O. & Knight, V. Axelrod-python/tourexec: v0.3.1 (2017-09-19). https://doi.org/10.5281/zenodo.896461 (2017).

[40] Campbell, O., Knight, V. & Harper, M. Axelrod-Python/axelrod-fortran: v0.3.1 (2017-08-04). https://doi.org/10.5281/zenodo.838980 (2017).

[41] Guyon, I., Weston, J., Barnhill, S. & Vapnik, V. Gene selection for cancer classification using support vector machines. *Machine learning* **46**, 389–422 (2002).

Extended Data

	Author	Average Score	Reproduced Rank	Original Rank	Average Cooperation Rate
k92r	Anatol Rapoport	2.878	1	1	0.922
k61r	Danny C Champion	2.791	12	2	0.954
k42r	Otto Borufsen	2.861	2	3	0.916
k49r	Rob Cave	2.826	4	4	0.892
k44r	William Adams	2.826	3	5	0.882
k60r	Jim Graaskamp and Ken Katzen	2.810	9	6	0.844
k41r	Herb Weiner	2.811	8	7	0.865
k75r	Paul D Harrington	2.826	5	8	0.802
k84r	T Nicolaus Tideman and Paula Chieruzz	2.812	7	9	0.882
k32r	Charles Kluepfel	2.817	6	10	0.873
k35r	Abraham Getzler	2.801	11	11	0.888
k68r	Fransois Leyvraz	2.808	10	12	0.917
k72r	Edward C White Jr	2.776	13	13	0.924
k46r	Graham J Eatherley	2.772	14	14	0.948
k83r	Paul E Black	2.743	15	15	0.935

Extended Data Table 1. Top 15 strategies in the reproduced tournament. The table lists the top 15 strategies from Axelrod's original tournament along with their rankings in our tournament. For each strategy we also report the average score and average cooperation rate in the reproduced tournament.

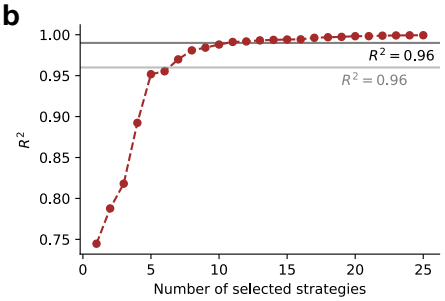
```
      FUNCTION K61R(ISPICK,ITURN,K,L,R, JA)
C BY DANNY C. CHAMPION
C TYPED BY JM 3/27/79
      k61r=ja     ! Added 7/27/93 to report own old value
      IF (ITURN .EQ. 1) ICOOP = 0  ! Added 10/8/2017 to fix bug for multiple runs
      IF (ITURN .EQ. 1) K61R = 0
      IF (ISPICK .EQ. 0) ICOOP = ICOOP + 1
      IF (ITURN .LE. 10) RETURN
      K61R = ISPICK
      IF (ITURN .LE. 25) RETURN
      K61R = 0
      COPRAT = FLOAT(ICOOP) / FLOAT(ITURN)
      IF (ISPICK .EQ. 1 .AND. COPRAT .LT. .6 .AND. R .GT. COPRAT)
     +K61R = 1
      RETURN
      END
```

Extended Data Fig 1. Fortran source code for `k61r` (Champion). The original implementation contained a bug: the variable `ICOOP` was not initialized within the function. In the first match it effectively started at 0, but in subsequent matches it could have kept its value from the previous match or be reset to 0, depending on the compiler and runtime environment. Line 5 shows the fix, which explicitly initializes `ICOOP` at the start of each match.

a

Strategies	Originally Reported Coefficients	Replicated Coefficients	p-value	F-value
k60r (S_6)	0.202000	0.232010	0.000000	298.163936
k91r (Revered State Transition)	0.198000	0.187930	0.000000	56.667196
k40r (Ruler)	0.110000	0.071870	0.000000	78.191831
k76r (Tester)	0.072000	0.063670	0.025992	5.207451
k67r (Tranquilizer)	0.086000	0.113650	0.000002	27.893148
Intercept	0.795000	0.790880	NA	NA

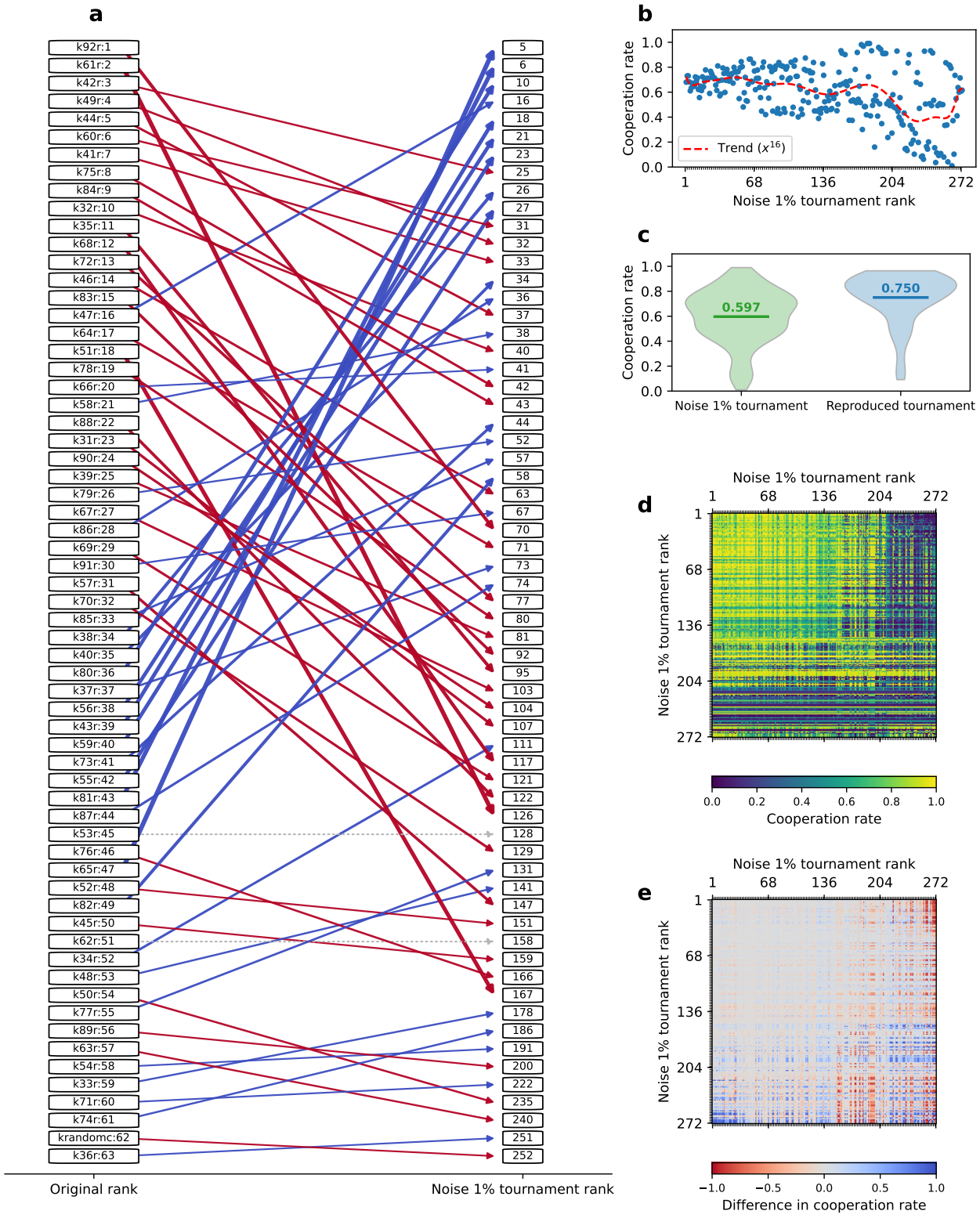
b



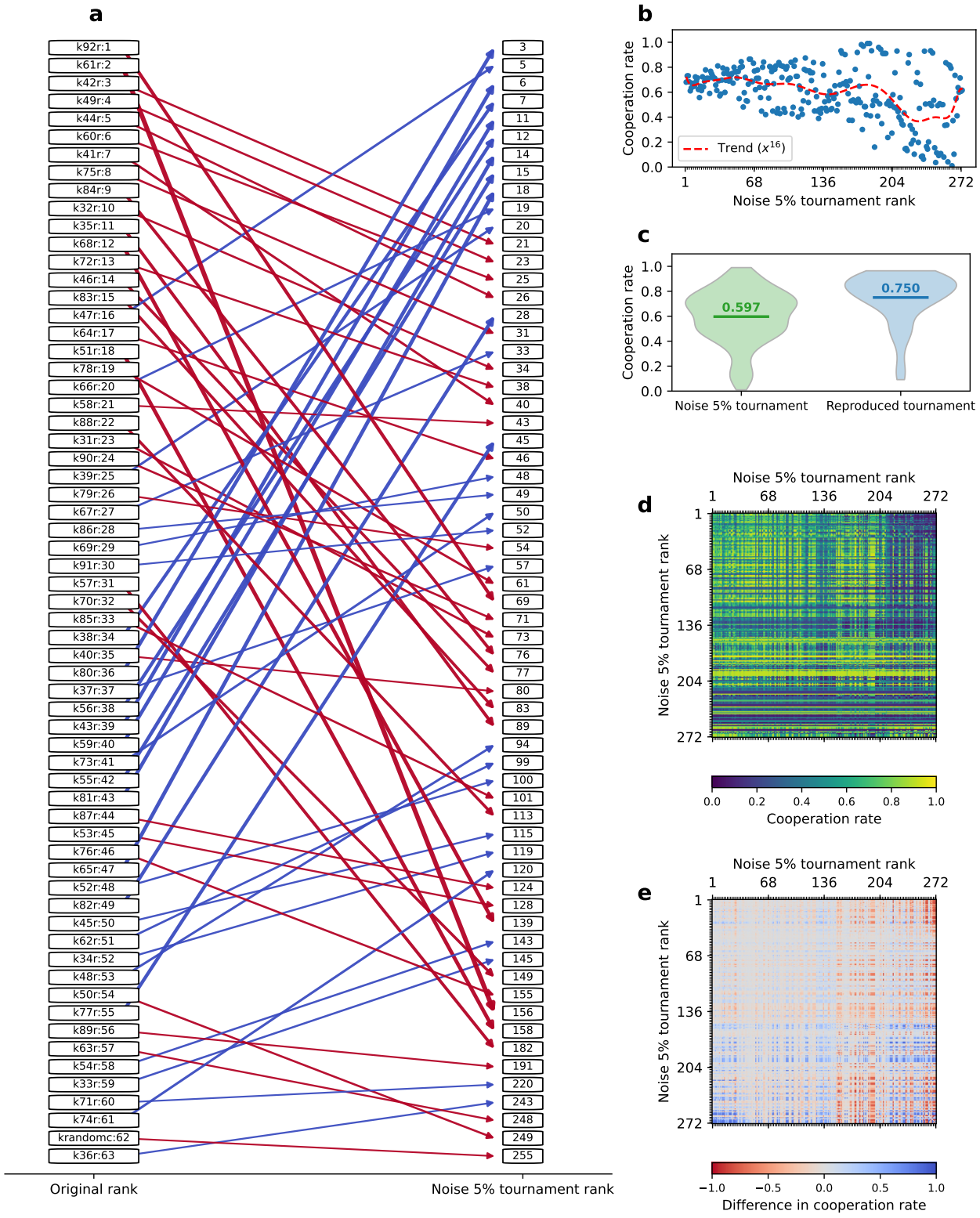
Extended Data Fig. 2. Reproduction of Axelrod's linear model. a, The representative strategies and their effect on average score. We reproduce the linear model described in [23], which reports $R^2 = 0.979$. Fitting a new model to the same five strategies yields $R^2 = 0.957$. **b,** R^2 as a function of the number of strategies included (subset size), obtained using recursive feature elimination (RFE) [41]. Starting from the full set of strategies, we iteratively remove the least informative one, refit the model, and record the resulting R^2. For each subset size (from 1 to 25), the subset yielding the highest R^2 is retained. The estimated R^2 rises rapidly from about 0.75 to 0.95 as the number of included strategies increases from 1 to 5, after which improvements are smaller.

Name	Average Score	Rank	Average Cooperation Rate	Reproduced Average Cooperation Rate	S. and P. Average Cooperation Rate
ZD-GTFT-2: 0.25, 0.5	2.834	1	0.926	-	0.855
k42r	2.825	2	0.903	0.916	-
GTFT: 0.33	2.819	3	0.941	-	0.893
k92r	2.812	4	0.888	0.922	0.731
k49r	2.787	5	0.879	0.892	-
k75r	2.786	6	0.782	0.802	-
k44r	2.781	7	0.866	0.882	-
k61r	2.773	8	0.945	0.954	-
k68r	2.768	9	0.904	0.917	-
k41r	2.761	10	0.847	0.865	-
k46r	2.758	11	0.939	0.948	-
k32r	2.758	12	0.849	0.873	-
k72r	2.756	13	0.915	0.924	-
k35r	2.747	14	0.863	0.888	-
k84r	2.745	15	0.861	0.882	-

Extended Data Table 2. Top 15 strategies in the Stewart and Plotkin tournament. We report the mean score, rank, and mean cooperation rate. For comparison, we also include the cooperation rate from our reproduced tournament, alongside the cooperation rates originally reported by Stewart and Plotkin.



Extended Data Fig. 3. Reproduction of Axelrod's second tournament with the addition of the strategies from the `Axelrod-Pythonand` with noise 1%. Similar to Figure 5.



Extended Data Fig. 4. Reproduction of Axelrod's second tournament with the addition of the strategies from the `Axelrod-Pythonand` with noise 5%. Similar to Figure 5.

	Average Score	Rank	Average Cooperation Rate	Original Rank	Reproduced Average Cooperation Rate	Library Rank	Library Average Cooperation Rate
EvolvedLookerUp2_2_2†	2.802	1	0.756	-	-	1	0.736
Evolved HMM 5†	2.788	2	0.751	-	-	2	0.742
Omega TFT: 3, 8	2.786	3	0.750	-	-	12	0.725
Evolved ANN 5†	2.784	4	0.720	-	-	3	0.713
Evolved ANN†	2.782	5	0.739	-	-	5	0.727
Evolved FSM 16†	2.780	6	0.731	-	-	4	0.713
Evolved FSM 16 Noise 05†	2.776	7	0.726	-	-	6	0.717
PSO Gambler 2_2_2†	2.760	8	0.702	-	-	7	0.692
Original Gradual	2.757	9	0.808	-	-	-	-
PSO Gambler 2_2_2 Noise 05†	2.756	10	0.740	-	-	14	0.723
PSO Gambler Mem1†	2.756	11	0.734	-	-	9	0.728
Evolved FSM 4†	2.752	12	0.793	-	-	10	0.777
PSO Gambler 1_1_1†	2.752	13	0.704	-	-	8	0.702
Gradual	2.746	14	0.763	-	-	15	0.793
DBS: 0.75, 3, 4, 3, 5	2.739	15	0.750	-	-	11	0.739
k42r	2.735	16	0.820	3	0.916	-	-

Extended Data Table 3. Top 16 strategies in the `Axelrod-Python` tournament. This tournament includes all strategies from the `Axelrod-Python` library alongside the original Fortran strategies. In total, there are 272 strategies competing. Strategies denoted by † were not designed but, they were trained via reinforcement learning, as described in [26].